

Proyecto: Lenguaje de Entrada

Brayan Esteban Alpízar Elizondo
Mariana González Sanabria
Frederick Obando Solano
Diego Salas Ovarés

Compiladores e Interpretes
Instituto Tecnológico de Costa Rica

4 de mayo de 2026

1. Especificación de la Arquitectura

1.1. Tamaño de palabra e instrucción

El ISA utiliza un tamaño de palabra de **32 bits**, tanto para datos como para instrucciones. Cada instrucción ocupa exactamente una palabra de memoria, lo que simplifica el diseño del hardware y permite una decodificación eficiente en arquitecturas tipo RISC.

1.2. Conjunto de registros

Para el sistema se definen dos bancos de registros:

✓ Banco principal (32 registros, 5 bits):

- zero: constante 0 (solo lectura)
- ra: dirección de retorno
- sp: puntero de pila
- pc: contador de programa
- lr: estado de sesión segura
- p0--p8: parámetros y retorno de funciones
- r0--r15: propósito general
- delta, max: constantes

✓ Banco seguro (8 registros, 3 bits):

- ax--hx: registros de propósito general protegidos

1.3. Organización de Memoria

La arquitectura objetivo del compilador utiliza una memoria direccionable total de **64 KB**, es decir:

$$64 \text{ KB} = 65\,536 \text{ bytes} = 0x10000 \text{ bytes}$$

Por tanto, todo el programa compilado debe caber dentro de ese espacio, incluyendo código, datos globales y memoria de pila.

Organización general

La memoria se organiza en los siguientes segmentos:

- ✓ **Segmento de código:** contiene las instrucciones ensamblador generadas por el compilador.
- ✓ **Segmento de datos globales:** contiene variables globales y demás datos estáticos del programa.
- ✓ **Segmento de pila (stack):** contiene variables locales, arreglos locales, parámetros que deban guardarse temporalmente y datos de activación de funciones.
- ✓ **Heap:** en la versión actual del compilador **no se implementa heap** ni reserva dinámica de memoria.

Distribución de memoria

La distribución real se calcula al finalizar la generación de ensamblador:

1. Primero se genera todo el **código**.
2. Luego, el segmento de **datos globales** se ubica inmediatamente después del código, alineado a palabra.
3. Finalmente, la base de la **pila** se coloca después de los datos globales, también alineada.

De esta manera se evita traslape entre código, datos y pila. Además, el compilador valida que el tamaño total utilizado no exceda los **65536** bytes disponibles.

Asignación de direcciones

Variables globales Las variables globales reciben una dirección absoluta dentro del segmento de datos. Su dirección final no se fija de forma definitiva al inicio, sino que se reajusta una vez conocido el tamaño final del código generado.

Funciones Cada función recibe una dirección dentro del segmento de código. Esa dirección corresponde a la posición de su etiqueta de entrada en el ensamblador generado.

Variables locales Las variables locales se almacenan dentro del **frame** de activación de su función. El compilador reserva espacio en la pila para:

- ✓ dirección de retorno,
- ✓ variables locales escalares,
- ✓ arreglos locales,
- ✓ copias temporales de parámetros cuando se requiere.

Cada variable local recibe un desplazamiento dentro de ese frame.

Parámetros Los primeros parámetros de una función se pasan mediante registros de parámetro ($p_0, p_1, \dots$). Si un parámetro necesita preservarse en memoria o tratarse como base direccionable, el compilador le reserva espacio adicional dentro del frame.

Manejo del espacio

El compilador calcula el tamaño de cada objeto según su tipo:

- ✓ `int, bool`: 4 bytes
- ✓ `char`: 1 byte
- ✓ punteros: 4 bytes
- ✓ arreglos: tamaño del elemento multiplicado por la cantidad de elementos
- ✓ `vault[n]`: se reserva como un bloque contiguo de n palabras, es decir, $4n$ bytes

Todos los espacios de memoria relevantes se alinean a palabra cuando corresponde, con el fin de mantener consistencia con la ISA y simplificar los accesos.

Gestión de pila

La pila se usa únicamente para almacenamiento temporal de ejecución. En esta implementación:

- ✓ al entrar a una función, se reserva su frame en pila;
- ✓ al salir, ese espacio se libera completamente;
- ✓ la pila se utiliza para variables locales, arreglos locales y soporte de llamadas.

Además, el compilador verifica que el frame más grande posible, junto con código y datos globales, siga cabiendo dentro de la memoria total de 64 KB.

En la versión desarrollada, la memoria queda organizada de forma **estática y controlada**:

- ✓ código al inicio,
- ✓ datos globales a continuación,
- ✓ pila después de los datos,
- ✓ sin heap dinámico.

Esta decisión simplifica el diseño del compilador y permite validar de manera temprana que el programa generado respete los límites de memoria de la arquitectura propuesta.

1.4. Tabla completa de instrucciones

Tabla 1. Tabla completa de instrucciones con opcode, formato y ejemplos.

Instr.	Opcode	Formato	Descripción	Ejemplo
add	00000	R	Suma registros	add r1, r2, r3
sub	00000	R	Resta registros	sub r1, r2, r3
mul	00000	R	Multiplicación	mul r1, r2, r3
div	00000	R	División	div r1, r2, r3
and	00000	R	AND lógico	and r1, r2, r3
orr	00000	R	OR lógico	orr r1, r2, r3
xor	00000	R	XOR lógico	xor r1, r2, r3
addi	00001	I	Suma inmediata	addi r1, r2, 10
subi	00001	I	Resta inmediata	subi r1, r2, 5
ldw	00100	M	Carga palabra	ldw r1, 0(r2)
stw	00101	M	Almacena palabra	stw r1, 4(r2)
beq	01000	B	Salto si iguales	beq r1, r2, L1
bne	01000	B	Salto si distintos	bne r1, r2, L1
jal	01001	J	Salto con enlace	jal ra, func
call	01001	J/F	Llamada a función	call func
ret	–	R	Retorno	ret
nop	–	R	No operación	nop
login	10001	S	Activa modo seguro	login 0xBEEF0
quit	10001	S	Desactiva modo seguro	quit
padd	00010	PR	Suma segura	padd ax, bx, cx
paddi	00011	PI	Suma inmediata segura	paddi ax, bx, 1
ldvw	00110	V	Carga vault	ldvw ax, 0(bx)
stvw	00111	V	Store vault	stvw ax, 0(bx)
send	10000	T	Normal → seguro	send ax, r1
recv	10000	T	Seguro → normal	recv r1, ax

1.5. Observaciones

- ✓ Las instrucciones seguras requieren sesión activa mediante **login**.

- ✓ El bit P indica ejecución en modo seguro.
- ✓ Las pseudo-instrucciones son traducidas por el compilador.

Con base en lo desarrollado hasta este punto, el compilador ya cuenta con una implementación funcional y consistente de la mayor parte del conjunto de instrucciones descrito para la ISA, incluyendo las extensiones seguras y de **vault**. En particular, para las categorías **PR**, **PI** y **T**, todas las instrucciones mostradas en la especificación están contempladas en el backend, y todas salvo ya aparecen efectivamente en la salida ensamblador generada por los ejemplos actuales.

2. Especificación del Lenguaje de Entrada

2.1. Tabla de tokens

Categoría	Ejemplos
Palabras reservadas	int, bool, char, int*, bool*, char*, ret, void, if, elif, else, for, while, vault, true, false, traigase, main, continue
Operadores aritméticos	+ (suma), - (resta), * (multiplicación), / (división), ~ (potencias), % (módulo)
Operadores lógicos	& (and), (or), ^ (xor), ! (not), « (shiftL), » (shiftR)
Identificadores de funciones y variables	func (funciones), int, bool, char (variables)
Operadores para comentarios	#, #* *#
Delimitadores	{, }, (,), [,], :, ,(coma)
Comparación	==, <, >, <=, >=, !=
Asignación	=, +=, -=, *=, /=, %=, &=, —=, ^=
Conjuntos de datos	Tipo_dato[longitud], po_dato[longitud1][longitud2]

Nota: El uso de la palabra reservada **vault** es exclusivo para la arquitectura usada en el proyecto de Arquitectura de Computadores I

2.2. Sintaxis de variables, funciones y estructuras de control.

2.2.1. Declaración de variable

```
<tipo> <variable>;      # datos sencillos
<tipo>* <variable>;      # punteros
<tipo> <variable>[<tamano>]; # arreglos
<tipo> <variable>[<tamano>][<tamano2>]; # matriz
```

2.2.2. Asignación de variables

```
# asignacion de datos
<tipo> <variable> = <variable o inmediato>;
<variable> = <variable o inmediato>;
<variable>[<valor>] = <variable o inmediato>; # en matrices

# asignacion de punteros
<variable> = &<variable>;
<tipo>* <variable> = &<variable>;

# asignacion con operacion -> op (+,-,*,/,%,&|,^)
<variable> <op>= <variable o inmediato>;
<variable>[<valor>] <op>= <variable o inmediato>; # en matrices
```

2.2.3. Definición de funciones

```
func <tipo> nombre_de_funcion(<parametros>){

    ret <variable>;
}
```

2.3. Ciclos

```
# Ciclos FOR
for (int <variable> = <inicio>; <variable> <op>= <valor>; <condicion>) {

    <codigo>

}

# Ciclos WHILE
while (<condicion>) {

    <codigo>

}
```

2.3.1. Condicionales

```
# Las condiciones deben de ir definidas con un comparador (==, !=, <,>, <=, >=)
if (<condicion>){
<codigo>
} elif (<condicion2>){
<codigo>
} else {
<codigo>
}

# Condiciones
<variable o inmediato> == <variable o inmediato> # igual a
<variable o inmediato> != <variable o inmediato> # diferente a
<variable o inmediato> <= <variable o inmediato> # menor o igual a
<variable o inmediato> >= <variable o inmediato> # mayor o igual a
<variable o inmediato> < <variable o inmediato> # menor a
<variable o inmediato> > <variable o inmediato> # mayor a
```

2.4. Definición de comentarios

```
# Comentarios de linea
**
Comentarios de
multiples lineas
**
```

2.5. Anotaciones o pragmas

Las anotaciones o pragmas son construcciones especiales del lenguaje que permiten agregar información adicional al compilador sin alterar la sintaxis básica de expresiones, funciones o estructuras de control. Su propósito es modificar o complementar el comportamiento de compilación, activando reglas, verificaciones o transformaciones específicas sobre una sección del programa.

En este lenguaje, los pragmas se utilizan para indicar contextos especiales de ejecución. Un pragma no representa una operación aritmética ni una instrucción convencional del programa, sino una directiva para el compilador. Por ello, su efecto principal ocurre durante el análisis semántico y la generación de código.

2.5.1. Pragma @secure

El pragma `@secure` se utiliza para indicar que un bloque de código debe compilarse bajo el contexto de seguridad. Su presencia señala al compilador que las instrucciones generadas dentro de dicho bloque pertenecen a una región protegida y, por tanto, deben ejecutarse con el mecanismo de protección activado.

La sintaxis general del pragma es la siguiente:

```
@secure(<identificador_hexadecimal>)
```

donde `<identificador_hexadecimal>` representa un valor de control de seguridad asociado al bloque protegido. Este valor, el cual funciona como una password, se utiliza como parámetro para habilitar el contexto seguro durante la traducción a código ensamblador.

2.5.2. Semántica del pragma

Cuando el compilador encuentra un pragma `@secure`, debe interpretar que el bloque inmediatamente asociado se ejecutará en modo seguro. Esto implica que:

1. Se activa un contexto de seguridad antes de emitir las instrucciones protegidas.
2. Las instrucciones correspondientes al bloque marcado se generan como instrucciones seguras.
3. Al finalizar el bloque, el compilador debe emitir la instrucción que desactiva el contexto seguro.

En términos de generación de código, el pragma `@secure` produce una transición entre dos estados de ejecución:

- ✓ **Estado normal:** el bit o indicador de protección tiene valor $P = 0$.
- ✓ **Estado seguro:** el bit o indicador de protección tiene valor $P = 1$.

Por tanto, el compilador debe generar una secuencia en la que se entra al modo seguro antes de las instrucciones protegidas y se sale de él al terminar el bloque.

2.5.3. Ejemplo en alto nivel

```
@secure(0xDEAD0)
func int foo(int a, int b){
    int x = (a + b) * 2;
    ret x;
}
```



```
int f = foo(1,2);
```

En este ejemplo, la función `foo` queda asociada a un contexto seguro identificado por el valor hexadecimal `0xDEAD0`. El compilador deberá generar código de entrada al modo seguro antes de las operaciones internas protegidas de la función y restaurar el modo normal al finalizar.

2.5.4. Traducción conceptual a ensamblador

Una posible traducción del ejemplo anterior al nivel ensamblador es la siguiente:

```
foo:
    addi sp, sp, 8
    str r5, 0(sp)
    str r6, 4(sp)
    login 0xDEAD0
    @add r1, r1, r2
    @multi r1, r1, 2
    quit
    ldr r6, 4(sp)
    ldr r5, 0(sp)
    subi sp, sp, 8
    ret
```

En esta traducción:

- ✓ Las instrucciones del prólogo y epílogo de la función permanecen en modo normal, es decir, con $P = 0$.
- ✓ La instrucción `login 0xDEAD0` activa el contexto seguro.
- ✓ Las instrucciones prefijadas con `@`, como `@add` y `@multi`, se ejecutan dentro del contexto protegido, es decir, con $P = 1$.
- ✓ La instrucción `quit` desactiva el contexto seguro y retorna al modo normal.

2.5.5. Alcance del pragma

El pragma `@secure` aplica sobre el bloque o definición que aparece inmediatamente después de su declaración. Si precede a una función, entonces dicha función será compilada como una región segura. Si en futuras extensiones del lenguaje se permite aplicarlo a otros bloques, su alcance seguirá la misma regla: afectar únicamente al bloque siguiente.

2.5.6. Restricciones

Para mantener claridad y consistencia en la compilación, el pragma **@secure** debe cumplir las siguientes restricciones:

1. Debe estar seguido de un identificador hexadecimal válido.
2. Debe aparecer antes del bloque o función que desea proteger.
3. No modifica la sintaxis interna del bloque, sino únicamente su tratamiento semántico y su traducción a ensamblador.
4. El compilador puede prohibir pragmas **@secure** anidados para evitar ambigüedad en el manejo del contexto protegido.

2.6. Ámbito de bloque

Los bloques de código, como los condicionales, los ciclos y las funciones, están delimitados por corchetes {}, los cuales agrupan el conjunto de instrucciones que pertenecen a una misma estructura. Esto permite definir con claridad qué sentencias forman parte de cada bloque lógico dentro del programa. Por ejemplo, en una estructura condicional:

```
if (x <= 5){  
    x = 6;  
} else{  
    x = 1;  
}
```

En este caso, las instrucciones encerradas entre {} después de **if** se ejecutan cuando la condición **x <= 5** es verdadera, mientras que las instrucciones dentro del bloque de **else** se ejecutan cuando dicha condición es falsa.

Por otro lado, cada línea de código que representa una instrucción individual debe finalizar con punto y coma ;. Este símbolo indica el final de una sentencia y es obligatorio en elementos como la declaración de variables, las asignaciones de valores y las llamadas a funciones. Su uso correcto permite separar instrucciones y evita ambigüedades en la interpretación del código por parte del compilador.

3. Descripción de Algoritmos Clave

En esta sección se describen los algoritmos principales implementados en el compilador, desde el reconocimiento léxico hasta la generación de ensamblador y la resolución de direcciones.

3.1. Análisis léxico y reconocimiento de tokens

El análisis léxico se implementa mediante una gramática de ANTLR en el archivo del lexer. El algoritmo sigue un recorrido secuencial sobre la entrada fuente y agrupa caracteres en tokens según expresiones regulares y reglas explícitas del lenguaje.

El lexer reconoce, entre otros:

- ✓ palabras reservadas como `func`, `if`, `while`, `for`, `ret`, `traigase`, `@secure`;
- ✓ identificadores;
- ✓ literales enteros, hexadecimales, reales, booleanos, cadenas y caracteres;
- ✓ operadores aritméticos, lógicos, relacionales y de asignación;
- ✓ delimitadores como paréntesis, llaves, corchetes, comas y punto y coma;
- ✓ comentarios y espacios en blanco, los cuales se ignoran adecuadamente.

Además, el lexer incluye reglas de error para detectar:

- ✓ caracteres no válidos;
- ✓ operadores inválidos;
- ✓ comentarios no cerrados;
- ✓ literales reales mal formados.

En términos algorítmicos, el lexer aplica un patrón de **máxima coincidencia** sobre la entrada. En cada posición intenta reconocer el token válido más largo posible, lo clasifica y avanza al siguiente punto de lectura. Cuando no existe una coincidencia válida, genera un token de error que luego el driver transforma en un mensaje léxico legible.

3.2. Análisis sintáctico y construcción del AST

El análisis sintáctico se basa en una gramática ANTLR para el parser. Una vez tokenizada la entrada, el parser verifica que la secuencia de tokens respete la estructura del lenguaje: declaraciones, funciones, bloques, expresiones, ciclos, condicionales, llamadas, arreglos, `vault`, etc.

La construcción del AST no se hace directamente en la gramática, sino mediante un **visitor** que recorre el parse tree generado por ANTLR y construye nodos más compactos y útiles para las siguientes fases.

El proceso sigue estas etapas:

1. El parser genera un árbol sintáctico concreto (parse tree).
2. El visitor recorre ese árbol.

3. Cada regla relevante del parser se transforma en un nodo del AST, por ejemplo:

- ✓ programa completo;
- ✓ importaciones;
- ✓ funciones;
- ✓ declaraciones de variables;
- ✓ asignaciones;
- ✓ sentencias de control;
- ✓ llamadas;
- ✓ operaciones unarias y binarias;
- ✓ accesos indexados.

4. El AST resultante conserva información de línea y columna para reportar errores posteriores.

La ventaja de este algoritmo es que separa la representación sintáctica concreta del análisis posterior. El AST elimina detalles innecesarios del parse tree y deja una estructura más adecuada para semántica y generación de ensamblador.

3.3. Construcción de tablas de símbolos

La construcción de la tabla de símbolos se realiza durante el análisis semántico mediante un recorrido del AST. El algoritmo utiliza una estructura jerárquica de ámbitos (*scopes*) con relaciones padre-hijo.

El procedimiento general es:

1. Crear el ámbito global.
2. Registrar primero los elementos de alto nivel:
 - ✓ funciones;
 - ✓ variables globales.
3. Para cada función:
 - ✓ abrir un nuevo ámbito de función;
 - ✓ registrar parámetros;
 - ✓ crear y mantener su frame de memoria;
 - ✓ recorrer el bloque interno.
4. Para cada bloque anidado:
 - ✓ abrir un ámbito hijo;

- ✓ registrar variables locales;
- ✓ cerrar el ámbito al finalizar el bloque.

Cada símbolo registrado almacena propiedades como:

- ✓ nombre;
- ✓ clase (*variable*, *function*, *parameter*, *label*);
- ✓ tipo;
- ✓ ámbito;
- ✓ segmento de memoria;
- ✓ dirección absoluta o desplazamiento;
- ✓ tamaño;
- ✓ registro asociado, si aplica.

La resolución de nombres se hace buscando primero en el ámbito actual y luego ascendiendo por la cadena de padres hasta llegar al ámbito global. Esto permite soportar sombreado local y distinguir variables con el mismo nombre en distintos bloques.

3.4. Algoritmo de cálculo de saltos

El cálculo de saltos se realiza en la fase de generación de ensamblador. Durante la emisión inicial, el backend no conoce todavía todas las direcciones finales del programa, por lo que utiliza referencias diferidas mediante etiquetas simbólicas.

El algoritmo funciona así:

1. Durante el recorrido del AST, el generador emite:
 - ✓ instrucciones;
 - ✓ marcadores de etiqueta;
 - ✓ referencias simbólicas a etiquetas.
2. Cada etiqueta se almacena como un `LabelMarker`.
3. Cada salto o branch se guarda con una referencia `LabelRef`.
4. En la etapa de render final:
 - ✓ se recorre la secuencia emitida;
 - ✓ se calcula el índice de instrucción de cada etiqueta;

✓ se asigna dirección final a cada punto del código.

5. Para cada referencia a etiqueta se calcula el desplazamiento relativo con la fórmula:

$$\text{offset} = \text{instr}_{\text{destino}} - \text{instr}_{\text{actual}} + 1$$

6. El generador valida que el desplazamiento quepa en el rango permitido por la ISA:

✓ branches: inmediato relativo de 12 bits;

✓ jumps/calls: inmediato relativo de 21 bits.

Este algoritmo permite emitir instrucciones como **beq**, **bne**, **jmp** o **call** sin conocer de antemano la ubicación final del destino. Solo al final se reemplaza la etiqueta por el valor relativo correspondiente.

3.5. Algoritmo de resolución de referencias

La resolución de referencias aparece en dos niveles: semántico y de generación de código.

1. Resolución semántica de nombres Durante semántica, cuando aparece un identificador:

1. se consulta el ámbito actual;
2. si no se encuentra, se consulta el ámbito padre;
3. el proceso continúa hasta el ámbito global;
4. si no existe coincidencia, se reporta referencia no declarada.

Esto aplica para:

✓ variables;

✓ parámetros;

✓ funciones;

✓ labels semánticos internos.

2. Resolución de direcciones en ensamblador En codegen, una vez resuelto el símbolo semántico, el compilador transforma nombres en ubicaciones concretas:

- ✓ una variable global se transforma en una dirección absoluta del segmento de datos;
- ✓ una variable local se transforma en un desplazamiento relativo dentro del frame de pila;
- ✓ un parámetro se transforma en un registro `p0`, `p1`, ..., o en una posición de stack si excede el límite;
- ✓ una función se transforma en una etiqueta del segmento de código;
- ✓ un acceso a arreglo o `vault` se transforma en:

$$\text{direccion base} + (\text{indice} \times \text{tamano del elemento})$$

Cuando se trata de globales, el backend primero usa referencias diferidas de dirección y luego, al estabilizar el layout final de código y datos, reemplaza esas referencias por direcciones reales.

Recopilación

En conjunto, el compilador implementa una cadena de algoritmos bien separada:

- ✓ el lexer reconoce tokens por coincidencia máxima;
- ✓ el parser valida la gramática y construye un parse tree;
- ✓ el visitor transforma el parse tree en un AST compacto;
- ✓ el analizador semántico construye ámbitos, tabla de símbolos y valida tipos;
- ✓ el generador de ensamblador resuelve referencias, calcula offsets y materializa instrucciones finales.

Esta organización modular permitió desarrollar cada fase de manera incremental y mantener coherencia entre la representación del lenguaje, la semántica y la generación de código.

4. Ejemplos de Ejecución

4.1. Ejercicio 1: Cálculo de Factorial

4.1.1. Código Fuente Completo

```
int global_result;
func int factorial(int a){
    int resultado = 1;
```

```

    int i = 1;

    while (i <= a) {
        resultado = resultado * i;
        i += 1;
    }
    global_result = resultado;
    ret resultado;
}

func void main(){
    factorial(8);
}

```

4.1.2. Ensamblador Intermedio Generado

```

; Codigo ensamblador generado por FCC
; ISA base: F32IS (isa.md)
__init__:    # addr=0
    li sp, 4
    call 29    # entrada principal | -> main @ 124
__halt__:    # addr=8
    jmp -1     # -> __halt__ @ 8
factorial:    # addr=12
    addi sp, sp, 12
    stw ra, +0(sp)
    li r0, 1
    stw r0, +8(sp)
    li r0, 1
    stw r0, +4(sp)
factorial_while_cond_1:    # addr=36
    ldw r0, +4(sp)
    mov r1, p0
    bgt r0, r1, 9    # -> factorial_while_end_2 @ 84
    ldw r0, +8(sp)
    ldw r1, +4(sp)
    mul r0, r0, r1
    stw r0, +8(sp)
    li r0, 1
    ldw r1, +4(sp)
    add r1, r1, r0
    stw r1, +4(sp)

```



```

    jmp -12    # -> factorial_while_cond_1 @ 36
factorial_while_end_2:    # addr=84
    ldw r1, +8(sp)
    la r0, 0
    stw r1, +0(r0)
    ldw r1, +8(sp)
    nop      # espera load-use antes de mover retorno
    nop      # espera load-use antes de mover retorno
    mov p0, r1
    ldw ra, +0(sp)
    addi sp, sp, -12
    ret
main:    # addr=124
    addi sp, sp, 4
    stw ra, +0(sp)
    li r0, 8
    mov p0, r0
    call -33    # -> factorial @ 12
    ldw ra, +0(sp)
    addi sp, sp, -4
    ret

```

4.1.3. Representación Binaria en Hexadecimal

```

00400882
00008752
FFFF83D2
00C10882
0001060A
00103882
00813A0A
00103882
00413A0A
00413A08
0002BC80
00F724D0
00813A08
00413E08
00F73900
00813A0A
00103882
00413E08

```

```
00E7BC80
00413E0A
FFFF8112
00813E08
00003882
00073E0A
00813E08
00000080
00000080
00079480
00010608
FF410882
00008C80
00410882
0001060A
00803882
00071480
FFFE87D2
00010608
FFC10882
00008C80
```

4.1.4. Traza de Ejecución

- ✓ La ejecución inicia en `main`, donde se carga el valor 8 y se llama a `factorial(8)`.
- ✓ En `factorial`, se inicializan las variables locales: `resultado = 1` e `i = 1`.
- ✓ La condición `if (a < 0)` se evalúa como falsa porque `a = 5`.
- ✓ El ciclo `while (i <= a)` se ejecuta cinco veces:
 - Iteración 1: `resultado = 1 * 1 = 1`, luego `i = 2`.
 - Iteración 2: `resultado = 1 * 2 = 2`, luego `i = 3`.
 - Iteración 3: `resultado = 2 * 3 = 6`, luego `i = 4`.
 - Iteración 4: `resultado = 6 * 4 = 24`, luego `i = 5`.
 - Iteración 5: `resultado = 24 * 5 = 120`, luego `i = 6`.
- ✓ Al salir del ciclo, la función retorna 120 en `p0`.
- ✓ Este valor de retorno se almacena en la variable global `global_result`.

4.2. Ejercicio 2: Búsqueda en Arreglo

4.2.1. Código Fuente Completo

```
int found_index;

func int buscar(int objetivo){
    int datos[5];
    int i = 0;
    int posicion = -1;
    bool encontrado = false;

    datos[0] = 4;
    datos[1] = 1;
    datos[2] = 7;
    datos[3] = 9;
    datos[4] = 6;

    while (i < 5) {
        if (datos[i] == objetivo) {
            posicion = i;
            encontrado = true;
        }

        if (!encontrado) {
            i += 1;
        } else {
            i = 5;
        }
    }

    found_index = posicion;
    ret posicion;
}

func void main(){
    buscar(9);
}
```

4.2.2. Ensamblador Intermedio Generado

```
;Codigo ensamblador generado por FCC
;ISA base: F32IS (isa.md)
__init__:    # addr=0
    li sp, 4
    call 76   # entrada principal | -> main @ 312
__halt__:    # addr=8
    jmp -1    # -> __halt__ @ 8
buscar:      # addr=12
    addi sp, sp, 36
    stw ra, +0(sp)
    li r0, 0
    stw r0, +12(sp)
    li r0, 1
    sub r0, zero, r0
    stw r0, +8(sp)
    li r0, 0
    stw r0, +4(sp)
    li r0, 4
    addi r1, sp, 16
    li r2, 0
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
    li r0, 1
    addi r1, sp, 16
    li r2, 1
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
    li r0, 7
    addi r1, sp, 16
    li r2, 2
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
    li r0, 9
    addi r1, sp, 16
    li r2, 3
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
    li r0, 6
```

```

    addi r1, sp, 16
    li r2, 4
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
buscar_while_cond_1:    # addr=168
    ldw r0, +12(sp)
    li r1, 5
    bge r0, r1, 23      # -> buscar_while_end_2 @ 272
    addi r1, sp, 16
    ldw r2, +12(sp)
    muli r2, r2, 4
    add r1, r1, r2
    ldw r0, +0(r1)
    mov r1, p0
    bne r0, r1, 5       # -> buscar_if_else_4 @ 228
    ldw r0, +12(sp)
    stw r0, +8(sp)
    li r0, 1
    stw r0, +4(sp)
    jmp 0      # -> buscar_if_end_3 @ 228
buscar_if_else_4:      # addr=228
buscar_if_end_3:      # addr=228
    ldw r0, +4(sp)
    seqz r0, r0
    beqz r0, 5      # -> buscar_if_else_6 @ 260
    li r0, 1
    ldw r1, +12(sp)
    add r1, r1, r0
    stw r1, +12(sp)
    jmp 2      # -> buscar_if_end_5 @ 268
buscar_if_else_6:      # addr=260
    li r1, 5
    stw r1, +12(sp)
buscar_if_end_5:      # addr=268
    jmp -26      # -> buscar_while_cond_1 @ 168
buscar_while_end_2:    # addr=272
    ldw r1, +8(sp)
    la r0, 0
    stw r1, +0(r0)
    ldw r1, +8(sp)
    nop      # espera load-use antes de mover retorno
    nop      # espera load-use antes de mover retorno
    mov p0, r1

```

```

    ldw ra, +0(sp)
    addi sp, sp, -36
    ret
main:    # addr=312
    addi sp, sp, 4
    stw ra, +0(sp)
    li r0, 9
    mov p0, r0
    call -80    # -> buscar @ 12
    ldw ra, +0(sp)
    addi sp, sp, -4
    ret

```

4.2.3. Representación Binaria en Hexadecimal

```

00400882
00020712
FFFF83D2
02410882
0001060A
00003882
00C13A0A
00103882
00E038C0
00813A0A
00003882
00413A0A
00403882
01013C82
00004082
00484102
0107BC80
0007BA0A
00103882
01013C82
00104082
00484102
0107BC80
0007BA0A
00703882
01013C82
00204082

```

00484102
0107BC80
0007BA0A
00903882
01013C82
00304082
00484102
0107BC80
0007BA0A
00603882
01013C82
00404082
00484102
0107BC80
0007BA0A
00C13A08
00503C82
00F75D50
01013C82
00C14208
00484102
0107BC80
0007BA08
0002BC80
00F71490
00C13A08
00813A0A
00103882
00413A0A
00000012
00413A08
00073A80
00071450
00103882
00C13E08
00E7BC80
00C13E0A
00000092
00503C82
00C13E0A
FFFF0192
00813E08
00003882
00073E0A

```
00813E08
00000080
00000080
00079480
00010608
FDC10882
00008C80
00410882
0001060A
00903882
00071480
FFFD8412
00010608
FFC10882
00008C80
```

4.2.4. Traza de Ejecución

- ✓ La ejecución inicia en `main`, donde se llama a `buscar(7)`.
- ✓ En `buscar`, se inicializa el arreglo local `datos = [4,1,7,9,6]`.
- ✓ También se inicializan las variables:
 - `i = 0`
 - `posicion = -1`
 - `encontrado = false`
- ✓ El ciclo `while (i <5)` revisa el arreglo elemento por elemento:
 - `i = 0`: se compara `datos[0] = 4` con 7; no coincide.
 - `i = 1`: se compara `datos[1] = 1` con 7; no coincide.
 - `i = 2`: se compara `datos[2] = 7` con 7; coincide.
- ✓ Cuando encuentra el valor:
 - `posicion = 2`
 - `encontrado = true`
 - se fuerza la salida del ciclo asignando `i = 5`
- ✓ La función retorna 2.
- ✓ El valor retornado se almacena en la variable global `found_index`.

4.3. Ejercicio 3: Fibonacci

4.3.1. Código Fuente Completo

```
# Programa propuesto: calculo de la serie de Fibonacci.
# Genera F(0)..F(10) en memoria y guarda F(10) como resultado final.

int fibonacci_series[11];
int fibonacci_result;

func int fibonacci_aux(int limite){
    int anterior = 0;
    int actual = 1;
    int siguiente = 0;
    int i = 2;

    if (limite < 0) {
        limite = 0;
    }

    if (limite > 10) {
        limite = 10;
    }

    fibonacci_series[0] = anterior;
    fibonacci_result = anterior;

    if (limite >= 1) {
        fibonacci_series[1] = actual;
        fibonacci_result = actual;
    }

    while (i <= limite) {
        siguiente = anterior + actual;
        fibonacci_series[i] = siguiente;

        anterior = actual;
        actual = siguiente;
        fibonacci_result = siguiente;
        i += 1;
    }

    ret fibonacci_result;
}
```

```

func int fibonacci(int n){
    int resultado = fibonacci_aux(n);
    ret resultado;
}

func void main(){
    fibonacci(10);
}

```

4.3.2. Ensamblador Intermedio Generado

```

; Codigo ensamblador generado por FCC
; ISA base: F32IS (isa.md)
__init__:    # addr=0
    li sp, 48
    call 93   # entrada principal | -> main @ 380
__halt__:    # addr=8
    jmp -1    # -> __halt__ @ 8
fibonacci_aux: # addr=12
    addi sp, sp, 20
    stw ra, +0(sp)
    li r0, 0
    stw r0, +16(sp)
    li r0, 1
    stw r0, +12(sp)
    li r0, 0
    stw r0, +8(sp)
    li r0, 2
    stw r0, +4(sp)
    mov r0, p0
    li r1, 0
    bge r0, r1, 3    # -> fibonacci_aux_if_else_2 @ 76
    li r0, 0
    mov p0, r0
    jmp 0    # -> fibonacci_aux_if_end_1 @ 76
fibonacci_aux_if_else_2:    # addr=76
fibonacci_aux_if_end_1:    # addr=76
    mov r0, p0
    li r1, 10
    ble r0, r1, 3    # -> fibonacci_aux_if_else_4 @ 100
    li r0, 10

```

```

    mov p0, r0
    jmp 0    # -> fibonacci_aux_if_end_3 @ 100
fibonacci_aux_if_else_4:    # addr=100
fibonacci_aux_if_end_3:    # addr=100
    ldw r0, +16(sp)
    la r1, 0
    li r2, 0
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
    ldw r0, +16(sp)
    la r1, 44
    stw r0, +0(r1)
    mov r0, p0
    li r1, 1
    blt r0, r1, 10    # -> fibonacci_aux_if_else_6 @ 188
    ldw r0, +12(sp)
    la r1, 0
    li r2, 1
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
    ldw r0, +12(sp)
    la r1, 44
    stw r0, +0(r1)
    jmp 0    # -> fibonacci_aux_if_end_5 @ 188
fibonacci_aux_if_else_6:    # addr=188
fibonacci_aux_if_end_5:    # addr=188
fibonacci_aux_while_cond_7:    # addr=188
    ldw r0, +4(sp)
    mov r1, p0
    bgt r0, r1, 22    # -> fibonacci_aux_while_end_8 @ 288
    ldw r0, +16(sp)
    ldw r1, +12(sp)
    add r0, r0, r1
    stw r0, +8(sp)
    ldw r0, +8(sp)
    la r1, 0
    ldw r2, +4(sp)
    muli r2, r2, 4
    add r1, r1, r2
    stw r0, +0(r1)
    ldw r0, +12(sp)
    stw r0, +16(sp)

```

```

    ldw r0, +8(sp)
    stw r0, +12(sp)
    ldw r0, +8(sp)
    la r1, 44
    stw r0, +0(r1)
    li r0, 1
    ldw r1, +4(sp)
    add r1, r1, r0
    stw r1, +4(sp)
    jmp -25    # -> fibonacci_aux_while_cond_7 @ 188
fibonacci_aux_while_end_8:    # addr=288
    la r0, 44
    ldw r1, +0(r0)
    nop    # espera load-use antes de mover retorno
    nop    # espera load-use antes de mover retorno
    mov p0, r1
    ldw ra, +0(sp)
    addi sp, sp, -20
    ret
fibonacci:    # addr=320
    addi sp, sp, 8
    stw ra, +0(sp)
    mov r0, p0
    mov p0, r0
    call -82    # -> fibonacci_aux @ 12
    nop    # espera retorno de call antes de leer p0
    mov r0, p0
    stw r0, +4(sp)
    ldw r0, +4(sp)
    nop    # espera load-use antes de mover retorno
    nop    # espera load-use antes de mover retorno
    mov p0, r0
    ldw ra, +0(sp)
    addi sp, sp, -8
    ret
main:    # addr=380
    addi sp, sp, 4
    stw ra, +0(sp)
    li r0, 10
    mov p0, r0
    call -20    # -> fibonacci @ 320
    ldw ra, +0(sp)
    addi sp, sp, -4
    ret

```

4.3.3. Representación Binaria en Hexadecimal

```
03000882
00028752
FFFF83D2
01410882
0001060A
00003882
01013A0A
00103882
00C13A0A
00003882
00813A0A
00203882
00413A0A
0002B880
00003C82
00F70D50
00003882
00071480
00000012
0002B880
00A03C82
00F70D90
00A03882
00071480
00000012
01013A08
00003C82
00004082
00484102
0107BC80
0007BA0A
01013A08
02C03C82
0007BA0A
0002B880
00103C82
00F72910
00C13A08
00003C82
00104082
00484102
0107BC80
```

0007BA0A
00C13A08
02C03C82
0007BA0A
00000012
00413A08
0002BC80
00F758D0
01013A08
00C13E08
00F73880
00813A0A
00813A08
00003C82
00414208
00484102
0107BC80
0007BA0A
00C13A08
01013A0A
00813A08
00C13A0A
00813A08
02C03C82
0007BA0A
00103882
00413E08
00E7BC80
00413E0A
FFFF01D2
02C03882
00073E08
00000080
00000080
00079480
00010608
FEC10882
00008C80
00810882
0001060A
0002B880
00071480
FFFD0792
00000080

```
0002B880
00413A0A
00413A08
00000080
00000080
00071480
00010608
FF810882
00008C80
00410882
0001060A
00A03882
00071480
FFFF0712
00010608
FFC10882
00008C80
```

4.3.4. Traza de Ejecución

- ✓ La ejecución comienza en `main`.
- ✓ Dentro de `main` se llama a `fibonacci(10)`.
- ✓ La función `fibonacci` recibe `n = 10`.
- ✓ Luego se llama a `fibonacci_aux(10)`:
 - se inicializan las variables locales:
 - `anterior = 0`
 - `actual = 1`
 - `siguiente = 0`
 - `i = 2`
 - `limite = 10`
 - Se evalúa la condición `limite < 0`.
 - Como `10 < 0` es falso, no se modifica `limite`.
 - Se evalúa la condición `limite > 10`.
 - Como `10 > 10` es falso, no se modifica `limite`.
 - Se guarda el primer valor de la serie:
 - `fibonacci_series[0] = 0`
 - `fibonacci_result = 0`

- Se evalúa la condición `limite >= 1`.
 - Como `10 >= 1` es verdadero:
 - ◊ `fibonacci_series[1] = 1`
 - ◊ `fibonacci_result = 1`
- Luego inicia el ciclo `while (i <= limite)`.
- Como `i = 2` y `limite = 10`, el ciclo se ejecuta mientras `i <= 10`.
- Evolución del ciclo:
 - Iteración 1, con `i = 2`:
 - ◊ `siguiente = anterior + actual = 0 + 1 = 1`
 - ◊ `fibonacci_series[2] = 1`
 - ◊ `anterior = 1`
 - ◊ `actual = 1`
 - ◊ `fibonacci_result = 1`
 - ◊ `i = 3`
 - Iteración 2, con `i = 3`:
 - ◊ `siguiente = 1 + 1 = 2`
 - ◊ `fibonacci_series[3] = 2`
 - ◊ `anterior = 1`
 - ◊ `actual = 2`
 - ◊ `fibonacci_result = 2`
 - ◊ `i = 4`
 - Iteración 3, con `i = 4`:
 - ◊ `siguiente = 1 + 2 = 3`
 - ◊ `fibonacci_series[4] = 3`
 - ◊ `anterior = 2`
 - ◊ `actual = 3`
 - ◊ `fibonacci_result = 3`
 - ◊ `i = 5`
 - Iteración 4, con `i = 5`:
 - ◊ `siguiente = 2 + 3 = 5`
 - ◊ `fibonacci_series[5] = 5`
 - ◊ `anterior = 3`
 - ◊ `actual = 5`
 - ◊ `fibonacci_result = 5`
 - ◊ `i = 6`
 - Iteración 5, con `i = 6`:

- ◇ siguiente = $3 + 5 = 8$
 - ◇ fibonacci_series[6] = 8
 - ◇ anterior = 5
 - ◇ actual = 8
 - ◇ fibonacci_result = 8
 - ◇ i = 7
- Iteración 6, con i = 7:
 - ◇ siguiente = $5 + 8 = 13$
 - ◇ fibonacci_series[7] = 13
 - ◇ anterior = 8
 - ◇ actual = 13
 - ◇ fibonacci_result = 13
 - ◇ i = 8
- Iteración 7, con i = 8:
 - ◇ siguiente = $8 + 13 = 21$
 - ◇ fibonacci_series[8] = 21
 - ◇ anterior = 13
 - ◇ actual = 21
 - ◇ fibonacci_result = 21
 - ◇ i = 9
- Iteración 8, con i = 9:
 - ◇ siguiente = $13 + 21 = 34$
 - ◇ fibonacci_series[9] = 34
 - ◇ anterior = 21
 - ◇ actual = 34
 - ◇ fibonacci_result = 34
 - ◇ i = 10
- Iteración 9, con i = 10:
 - ◇ siguiente = $21 + 34 = 55$
 - ◇ fibonacci_series[10] = 55
 - ◇ anterior = 34
 - ◇ actual = 55
 - ◇ fibonacci_result = 55
 - ◇ i = 11
- Al volver a evaluar el ciclo:
 - i = 11
 - limite = 10

- la condición `11 <= 10` es falsa
 - por lo tanto, se sale del `while`
- La función `fibonacci_aux` retorna:
 - `fibonacci_result = 55`
- ✓ La función `fibonacci` recibe el valor retornado:
 - `resultado = 55`
 - luego retorna `resultado`
- ✓ Finalmente, la llamada `fibonacci(10)` termina y se han generado los valores:
 - `fibonacci_series[0] = 0`
 - `fibonacci_series[1] = 1`
 - `fibonacci_series[2] = 1`
 - `fibonacci_series[3] = 2`
 - `fibonacci_series[4] = 3`
 - `fibonacci_series[5] = 5`
 - `fibonacci_series[6] = 8`
 - `fibonacci_series[7] = 13`
 - `fibonacci_series[8] = 21`
 - `fibonacci_series[9] = 34`
 - `fibonacci_series[10] = 55`
- ✓ Resultado final:
 - `fibonacci_result = 55`, que en memoria se lee como 37, al estar en hexadecimal

5. Manejo de Casos Especiales

Además del flujo normal de compilación, el proyecto contempla varios casos especiales que deben resolverse correctamente para mantener consistencia semántica y generar ensamblador válido. A continuación se describe cómo se manejan esos escenarios en la implementación desarrollada.

5.1. Saltos anidados: condicionales dentro de ciclos

Los saltos anidados se resuelven mediante una combinación de:

- ✓ etiquetas semánticas y de ensamblador generadas de forma única;
- ✓ una pila interna de contextos de ciclo;

- ✓ saltos relativos diferidos que se resuelven al final del proceso de render.

Cuando el compilador entra a un ciclo **while** o **for**, crea etiquetas específicas para:

- ✓ condición del ciclo;
- ✓ actualización del ciclo, en el caso de **for**;
- ✓ salida del ciclo.

Además, se registra en una pila el par de destinos asociados a:

- ✓ **continue**: hacia la reevaluación o actualización del ciclo;
- ✓ **break**: hacia la salida del ciclo.

Cuando aparece un **if** dentro de un ciclo, el compilador genera etiquetas adicionales para:

- ✓ rama **else**;
- ✓ fin del **if**;
- ✓ posibles ramas **elif**.

Gracias a que cada bloque genera nombres únicos de etiqueta y a que los contextos de ciclo se apilan, un **break** o **continue** siempre se enlaza con el ciclo correcto, incluso cuando existen condicionales internos o múltiples niveles de anidamiento.

5.2. Llamadas a función y retorno de valores

Las llamadas a función se resuelven en dos etapas: semántica y generación de código.

Validación semántica Antes de generar ensamblador, el compilador verifica que:

- ✓ la función llamada exista;
- ✓ el número de argumentos coincida con la firma;
- ✓ los tipos de los argumentos sean compatibles;
- ✓ la llamada no intente invocar algo que no sea función.

Paso de parámetros Durante la generación de ensamblador:

- ✓ los argumentos se evalúan de izquierda a derecha;
- ✓ los primeros parámetros se colocan en registros `p0`, `p1`, ...;
- ✓ si hay registros temporales vivos durante la llamada, el compilador los guarda temporalmente en pila;
- ✓ si un parámetro necesita preservarse como base direccionable, puede reservarse espacio adicional dentro del frame.

Retorno El valor de retorno se entrega en `p0`. Si la función es `void`, no se espera valor útil de retorno.

Al ejecutar una sentencia `ret`:

- ✓ se evalúa la expresión de retorno, si existe;
- ✓ el resultado se mueve a `p0`;
- ✓ se restaura el estado necesario de la función;
- ✓ se ajusta el frame de pila;
- ✓ se emite `ret`.

En funciones seguras (`@secure`), además del valor de retorno se respeta la secuencia de seguridad correspondiente: `login`, validación de contexto, operaciones seguras, `quit` y salida de la función.

5.3. Conflictos entre variables con el mismo nombre en distintos ámbitos

El compilador maneja variables homónimas mediante una tabla de símbolos jerárquica basada en ámbitos anidados.

Cada vez que se entra a:

- ✓ una función;
- ✓ un bloque;
- ✓ una estructura de control con bloque propio;

se crea un nuevo ámbito hijo. Las definiciones se registran en el ámbito actual, y las búsquedas siguen esta estrategia:

1. buscar en el ámbito local actual;

2. si no se encuentra, buscar en el ámbito padre;
3. continuar hasta llegar al ámbito global.

Esto permite que una variable local o de bloque **sombree** a otra con el mismo nombre declarada en un ámbito superior. Por ejemplo, una variable dentro de un bloque puede ocultar temporalmente a una variable global o a una variable del bloque externo.

Sin embargo, dos símbolos con el mismo nombre **dentro del mismo ámbito** sí se consideran error y producen diagnóstico de redeclaración.

5.4. Asignación de memoria para variables locales y globales

La asignación de memoria depende del tipo de símbolo y del segmento donde reside.

Variables globales Las variables globales se almacenan en el segmento de datos. El proceso es:

1. se registran durante la fase semántica;
2. se les asigna una dirección provisional dentro del segmento global;
3. al finalizar la generación de código, sus direcciones se reubican según el tamaño final del segmento de código.

El tamaño reservado depende del tipo:

- ✓ `int`, `float`, `bool`: 4 bytes;
- ✓ `char`: 1 byte;
- ✓ arreglos: tamaño del elemento por número de elementos;
- ✓ `vault[n]`: bloque contiguo de $4n$ bytes.

Variables locales Las variables locales se asignan dentro del frame de activación de la función. Para ello:

- ✓ cada función inicia con un tamaño de frame propio;
- ✓ cada variable local recibe un desplazamiento negativo o relativo dentro del stack frame;
- ✓ arreglos locales y estructuras tipo `vault` reservan memoria contigua dentro del mismo frame.

Parámetros Los parámetros se ubican preferentemente en registros de parámetro. Si es necesario tratarlos como memoria direccionable o si exceden la convención de paso por registros, se les asigna espacio complementario en pila.

Validaciones de memoria La implementación también verifica que:

- ✓ los offsets de acceso a memoria estén dentro del rango permitido por la ISA;
- ✓ las direcciones globales finales no excedan el espacio de memoria total;
- ✓ el frame más grande, junto con código y datos, quepa dentro de los 64 KB de memoria direccionable.

Los casos especiales se resuelven mediante estructuras de soporte ya integradas al compilador:

- ✓ pilas de contexto para ciclos y saltos anidados;
- ✓ convención consistente para llamadas y retornos usando registros de parámetros;
- ✓ tabla de símbolos jerárquica para manejar sombreado y redeclaraciones;
- ✓ asignación diferenciada de memoria para globales, locales, parámetros y **vault**.

Esto permite que el compilador mantenga coherencia incluso cuando el programa fuente presenta anidamiento, reutilización de nombres y múltiples clases de almacenamiento.